

Voice Interface and Spoken Tool Use

Eigenständiges druckbares Deep-Dive-Dossier

Voice Interface and Spoken Tool Use

Ich wollte mit meinem System sprechen koennen, aber Sprache ist für Tools riskant: sie ist schnell, ungenau und kann trotzdem Aktionen auslösen. Deshalb brauchte ich Muting, Tool-Grenzen, Fallbacks und Schutz gegen gefährliche Befehle.

- Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.
- Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

Was ich mir dabei gedacht habe

Voice war spannend, weil sich das System dadurch direkt anfühlte. Genau deshalb ist es aber riskant. Wenn ich spreche und der Computer handelt, dann sind Mute, Bestätigung, Shell-Grenzen und Privacy kein Zubehör, sondern Teil des Produkts.

Was ich gebaut habe

- Ich habe Voice als tool-fähige Kontrolloberfläche gebaut, nicht nur als Chat-Spielerei.
- Ich habe Datenschutz, Bestätigung und Unterbrechbarkeit von Anfang an mitgedacht.
- Ich habe hochwertige Stimmen mit praktischer Tool-Nutzung verbunden.

Mechanik im Detail

- Das Voice-System ist nicht nur Speech-to-Text. Gesprochene Befehle können Tool Calls, Vault-Operationen, Daemon Calls und begrenzte Shell-Ausführung auslösen.
- Damit ist Voice eine sicherheitssensible Fläche. Mute, Fallbacks, Filter für gefährliche Befehle, Timeouts und Privacy-Grenzen sind Kernarchitektur.
- Der spätere Plan zu Wake Word, lokaler STT, hochwertiger TTS und Barge-in zeigt Bewusstsein für Latenz, Privacy und Unterbrechbarkeit.

Architektur

- Python streamt Mikrofon-Audio in Gemini Live.
- Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.
- Gefährliche Shell-Muster werden blockiert, Ausführung bekommt Timeouts.
- Spätere Planung umfasst Wake Word, lokale STT, Hybrid-TTS, Barge-in und Privacy-Grenzen.

Evidenzcluster

- Voice-Source mit Mikrofonstreaming, Gemini-Live-Konfiguration, Tool Declarations, Dispatcher, Vault Tools, Deep-Think-Route, begrenzter Shell und Fallback.
- Voice-Pipeline-Notizen zu Wake Word, lokaler STT, hybrider TTS, Barge-in, Latenzbudget und Privacy-Grenzen.
- VoiceBridge-Notizen zur Trennung von Browser/Orb-STT und lokaler Daemon-Antwortlogik.

- Sicherheitsrelevante Quellabschnitte zu Mute-Verhalten, Filterung gefährlicher Befehle, Timeouts und Tool-Dispatch.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.	Voice-Source mit Mikrofonstreaming, Gemini-Live-Konfiguration, Tool Declarations, Dispatcher, Vault Tools, Deep-Think-Route, begrenzter Shell und Fallback.; Voice-Pipeline-Notizen zu Wake Word, lokaler STT, hybrider TTS, Barge-in, Latenzbudget und Privacy-Grenzen.	Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Python streamt Mikrofon-Audio in Gemini Live.; Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.

- Realismus im Human Interface: Voice senkt Reibung so stark, dass Authority Checks stärker werden müssen.
- Privacy-Realismus: Always-listening-Systeme sollten lokal und mit klaren Grenzen arbeiten.
- Safety-Realismus: Barge-in ist Kontrollfunktion, kein Luxus.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Voice Interface and Spoken Tool Use.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

- Voice senkt Reibung beim Auslösen und muss deshalb Reibung beim Risiko erhöhen.
- Barge-in ist ein Sicherheitsfeature, weil ich das System sofort stoppen können muss.
- Eine schöne Stimme ist wertlos, wenn Autorität und Privacy unklar bleiben.

Nächste Härtung / nächster Projektschritt

- Öffentliche Demo ohne private Mikrofon-/Vault-Daten bauen.
- Safe Tools und confirmation-required Tools klar trennen.
- Transkript-Audit und Korrekturpfade ergänzen.
- Eine öffentliche Voice-Simulation mit Transkripten statt Live-Mikrofon bauen.
- Safe Tools von bestätigungspflichtigen Tools im UI trennen.
- Audit Trail ergänzen: gehörter Satz, gearberte Absicht, gewähltes Tool, blockierter Befehl, Bestätigung und finale Antwort.